InvertedBlock Indexing for Learned Sparse MIPS

Roger Canal Estrada[1⋆]

Universitat Politècnica de Catalunya (UPC)
Facultat d'Informàtica de Barcelona (FIB)
roger.canal02@gmail.com

Abstract. Learned Sparse Representations (LSR), such as SPLADE, preserve lexical interpretability but create high Maximum Inner Product Search (MIPS) latency in very high-dimensional sparse spaces. This paper presents a static `InvertedBlock` indexing architecture with two phases: a static index partitioned by document clustering, and a parameterized search based on Coordinate-at-a-Time (CaaT) traversal. The static phase preserves candidate coverage, while the dynamic phase controls the latency–recall trade-off through runtime constraints. On Natural Questions (2.68M documents, 30,522 dimensions), the system reaches a recall@30 ≥ 0.90 at around 40 ms average latency.

Keywords: Learned Sparse Retrieval · Approximate Nearest Neighbor Search · Dynamic Pruning · Inverted Indexing

1 Introduction

LSR models map semantics into vocabulary space, so retrieval remains sparse and interpretable [2]. However, LSR query weights do not follow the same heavy-head Zipfian pattern that classical lexical pipelines exploit. Many coordinates that would look uninformative in traditional lexical retrieval (for example punctuation or low-semantic tokens) still receive non-negligible learned weights and can affect ranking quality; this behavior is often discussed as *wacky weights* [4,5]. This geometry weakens the pruning selectivity of traditional WAND-like and Document-at-a-Time (DaaT) traversal because partial-score upper bounds become less decisive at early stages [1].

This work addresses this limitation with block-level reasoning. The key idea is to decompose each posting list into geometrically coherent blocks and score blocks through tight upper bounds before touching documents.

The paper contributes three elements. First, it defines a static *Inverted Block Index* architecture where the high-capacity index preserves coverage. Second, it enforces exact-maximum summary bounds at index construction, making block upper bounds mathematically safe for aggressive skipping. Third, it shows results that reaches a operating point above recall@30 = 0.90. Similar block-centric ideas have also been explored in recent LSR systems such as SEISMIC [6], which has motivated the exploration of this study.

⋆ Corresponding author: roger.canal02@gmail.com

2 System Architecture

The system is built around a custom partitioned inverted index designed to support efficient approximate Maximum Inner Product Search over sparse vectors. The central goal of this data structure is to organize document information so that the search engine can quickly identify the most similar documents to any incoming query without scoring every document individually.

The pipeline has two phases: static `InvertedBlock` construction (Section 2.1) and parameterized query-time search (Section 2.2).

2.1 Static Index Phase

The static index has three steps. First, sparse K-means assigns each document to exactly one cluster. Second, for every cluster and vocabulary term, the index computes exact maximum term weights to build dense summary vectors. Third, for each term, it keeps only the top-nb clusters and then the top-nd documents per retained cluster, producing the final `InvertedBlock` lists used at query time.

Document Clustering Step. The method partitions the document set $\mathcal{D} = \{d_i\}_{i=1}^{N}$ into K clusters by a sparse K-means objective:

$$\max_{\{\mathcal{C}_k\},\{\mu_k\}} \sum_{k=1}^{K} \sum_{d_i \in \mathcal{C}_k} \langle d_i, \mu_k \rangle, \tag{1}$$

with assignment step

$$a(i) = \arg \max_{k \in \{1,\dots,K\}} \langle d_i, \mu_k \rangle, \tag{2}$$

and centroid update

$$\mu_k \leftarrow \frac{\sum_{d_i \in \mathcal{C}_k} d_i}{\left\| \sum_{d_i \in \mathcal{C}_k} d_i \right\|_2}. \tag{3}$$

Both operations are computed in sparse form. This is a *hard clustering*: each document d_i is assigned to exactly one cluster $\mathcal{C}_{a(i)}$, and no document belongs to more than one cluster.

Partitioned Inverted Index Construction. The index maps each concept t to a list of `InvertedBlock`s. Each block stores one cluster id and truncated doc-weight pairs. For each cluster c, the method also builds a dense summary vector $S_c \in \mathbb{R}^{|V|}$ with

$$S_c[t] = \max_{d \in \mathcal{C}_c} d[t]. \tag{4}$$

This is the absolute maximum weight of concept t inside cluster c.

Algorithm 1 Static Index Construction with Exact-Maximum Summaries

Require: Sparse documents, cluster assignments, nb, nd
Ensure: Partitioned index I and summary vectors S

for each cluster c **do**
  initialize dense summary $S_c \leftarrow 0$
  for each document $d \in \mathcal{C}_c$ **do**
    for each non-zero (t, w) in d **do**
      $S_c[t] \leftarrow \max(S_c[t], w)$
    end for
  end for
end for
for each concept t **do**
  aggregate candidate docs by cluster and concept weight
  keep top-nb clusters by accumulated weight on t
  for each retained cluster c **do**
    keep top-nd docs by weight on t
    append `InvertedBlock`$(c, doc\_ids, weights)$ to $I[t]$
  end for
end for
return I, S

For each concept t, the index keeps at most nb cluster blocks and at most nd documents per retained block. Exact maxima make the cluster upper bound mathematically valid (never under-estimated).

$$UB_c(q) = \sum_{t \in \text{supp}(q)} q[t] \cdot S_c[t] \tag{5}$$

This property prevents miss-skipping. If summaries under-estimate, the system can discard clusters that contain true top-k documents.

2.2 Search Phase

Coordinate-at-a-Time (CaaT) Traversal. The search engine processes incoming sparse queries over the static index with CaaT traversal. [1] Algorithm 2 formalizes the CaaT loop.

The search engine exposes four hyperparameters that restrict traversal:

1. `heap_factor`: relaxation multiplier controlling skip confidence.
2. `max_query_terms` (mqt): truncates low-signal query tail coordinates.
3. `max_search_blocks` (msb): limits per-coordinate block breadth online.
4. `max_docs_to_visit` (md): hard computational budget bounding worst-case latency.

[1] CaaT is preferred over classical Document-at-a-Time (DaaT) traversal [1] because it visits the highest-signal query coordinates first, rapidly inflating the min-heap threshold. A high threshold amplifies subsequent block-skipping decisions. DaaT, by contrast, mixes low- and high-signal evidence across all documents before the threshold stabilizes, reducing early pruning power.

Algorithm 2 Parameterized CaaT Search

Require: Query q, index I, summaries S, top-k, Max Query Terms mqt, Max Search Blocks msb, $heap\_factor$, Max Docs md
Ensure: Top-k result heap H

$Q \leftarrow$ non-zero query coordinates sorted by weight descending
if $mqt > 0$ **then**
    truncate Q to first mqt terms
end if
compute $UB_c(q)$ for all clusters c
initialize min-heap H, visited set, $docs\_examined \leftarrow 0$
for each term $t \in Q$ **do**
    $blocks\_seen \leftarrow 0$
    for each block $b \in I[t]$ **do**
        if $msb > 0$ and $blocks\_seen \geq msb$ **then**
            break
        end if
        $blocks\_seen \leftarrow blocks\_seen + 1$
        if $UB_{b.cluster\_id}(q) < H.\min() \cdot heap\_factor$ **then**
            continue
        end if
        for each doc d in b **do**
            if d not visited **then**
                score $\leftarrow \langle q, d \rangle$ and update H
                mark visited; $docs\_examined \leftarrow docs\_examined + 1$
                if $md > 0$ and $docs\_examined \geq md$ **then**
                    return H
                end if
            end if
        end for
    end for
end for
return H

It parses non-zero query coordinates, sorts them by descending weight, and enters the inverted lists of each coordinate in turn. In practice, this means the algorithm front-loads the strongest query evidence, applies the query-coordinate cap when requested, and then uses the block upper bounds to skip low-promise regions before spending the document budget.

Aggressive Dynamic Pruning Heuristics. The pruning stage rejects blocks by the inequality:

$$UB_c(q) < H.\min() \cdot heap\_factor. \tag{6}$$

If the inequality holds, the block cannot improve the current top-k under the selected confidence multiplier. This rule replaces expensive document-level scoring with $O(1)$ float comparisons at block granularity.

3 Experimental Evaluation

3.1 Experimental Setup

The evaluation uses the SISAP 2026 Task 3 dataset: Natural Questions (NQ) with 2.68M documents encoded as SPLADE sparse vectors in 30,522 dimensions. Recall@30 is measured against the provided exact k-NN ground truth, and average query latency is reported. [2]

Experimental methodology. We evaluate the build-time and search-time parameters in separate sweeps so that each family can be interpreted against a fixed baseline. We first vary K while holding the search parameters fixed, then vary nb and nd on a selected K setting, and finally sweep the search parameters (md, mqt, and msb) on a small set of representative fixed index configurations. This design avoids chaining one optimum directly into the next sweep; each subsection states the exact values held constant in that experiment.

Hardware. All runs execute inside a Docker container constrained to 8 virtual CPUs and 24 GB RAM with swap disabled. The host machine is an AMD Ryzen 5 5600X (6 cores / 12 logical processors, base 3.70 GHz, 32 MB L3 cache). Minor rerun variation can still arise from runtime details such as thread scheduling and OpenMP placement (for example `OMP_NUM_THREADS`), so we report trends and operating ranges rather than over-interpreting a single point estimate.

3.2 Index Phase Experimentation

We first study the build-time parameters with the search phase held fixed. The resulting experiments isolate how the static index configuration shapes the recall ceiling before any search-time pruning is introduced. Because documents excluded at build time by Alg. 1 are permanently unrecoverable, these sweeps characterize the maximum attainable recall of any later search configuration. The goal is therefore to select a fixed index setup for the search-phase experiments that is both efficient and sufficiently permissive.

Effect of Cluster Count K. A dedicated sweep evaluates K from 50 to 3000 with fixed $itr = 3$, $nb = 500$, $nd = 500$, `heap_factor` $= 0.05$, $mqt = 0$, and $msb = 150$. The three md values, 50,000, 100,000, and 150,000, span a low, medium, and high document budget so that we can observe how cluster granularity interacts with increasingly permissive scoring limits. Figure 1 summarizes the trend.

The updated sweep shows the expected trade-off between coverage and efficiency. Small K values preserve recall better but require more work per query; for example, $K = 100$ reaches Recall@30 $= 0.989$ at 91.4 ms/query. Larger K values reduce latency by making each retained block more selective, but this comes with a modest recall loss because candidate mass is split across more

[2] The results shown correspond to commit `96dbbe68adbd486f7cdf7a716f708b93f8a78479`. Repository: `https://github.com/channelEs/seed_42-SISAP-26`.

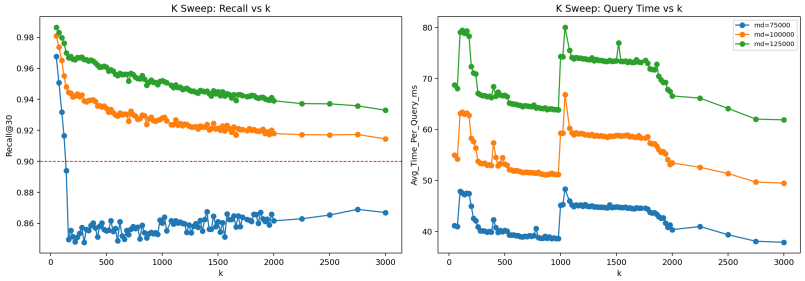

Fig. 1. Recall@30 and average query time as a function of K for three md values ($md = [50000, 100000, 150000]$).

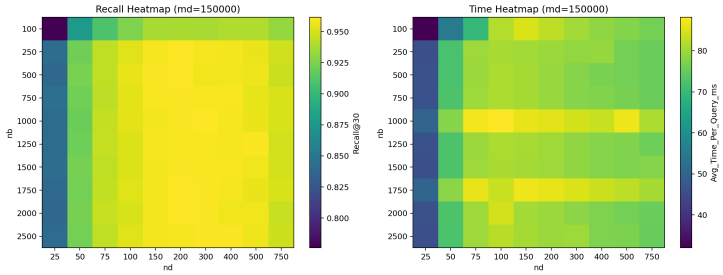

Fig. 2. Recall@30 heatmaps over nb and nd at $md = 150000$.

partitions under the same nb/nd truncation; for example, $K = 1800$ reaches Recall@30 = 0.954 at 76.6 ms/query. We therefore select $K = 1800$ as the best speed–quality compromise for the fixed-index search experiments.

Effect of InvertedBlock parameters (nb) and (nd). This sweep fixes $K = 1500$, $itr = 3$, $md = 150000$, `heap_factor`= 0.05, mqt= 0, and msb= 150, and varies nb and nd. Figure 2 summarizes the recall surface.

The results confirm two patterns. First, increasing nd from very low values, such as 25, to the 150–400 range substantially improves recall because each retained cluster block contributes more document evidence. Second, increasing nb beyond the medium range yields smaller and less stable gains, which indicates diminishing returns once the most informative clusters are already retained. The highest recall in this sweep is achieved at $nb = 1000$, $nd = 300$ (Recall@30 = 0.961), while the fastest configuration above roughly 0.96 recall is around $nb = 2500$, $nd = 400$ (77.4 ms/query). These results justify fixing a moderately permissive nb/nd setting for the search-phase experiments rather than pushing either parameter to its maximum.

3.3 Search Parameterization and Latency Optimization

In this section, the static index is fixed to the 02 baseline configuration: $K = 1800$, $itr = 3$, $nb = 500$, and $nd = 400$ (from `baseline_fixed_indexing.json`).

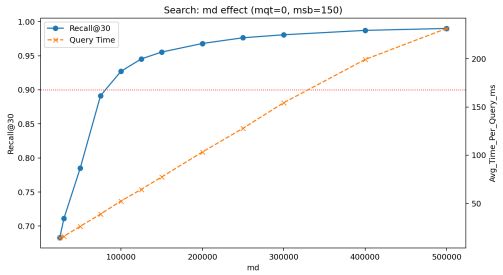

Fig. 3. Effect of *md* on Recall@30 and query latency for the fixed 02 index ($K = 1800$, $nb = 500$, $nd = 400$). Both metrics grow monotonically. Recall@30 ≥ 0.90 is first met around $md = 100000$.

On top of this fixed index, the search parameters are swept independently. Algorithm 2 governs all four parameters.

In this section, *md* denotes the *Maximum number of Documents* that can be fully scored per query.

Maximum number of Documents (*md*). *md* is the strongest recall lever. This sweep fixes `heap_factor`= 0.05, mqt= 0, and msb= 150, and varies *md* from 25,000 to 500,000. Figure 3 shows the resulting trend.

Recall and latency grow monotonically with *md*: in the baseline runs, recall increases from 0.683 at $md = 25{,}000$ to about 0.927 at $md = 100{,}000$ and to about 0.955 at $md = 150{,}000$, with average query time rising from about 14.5 ms to about 51.6 ms and 77.4 ms, respectively. This confirms that *md* is the primary recall–latency control under this operating regime.

Max Query Terms (mqt). mqt limits how many of the highest-weighted query coordinates are *included* in traversal (mqt = 0 means no truncation, i.e., all non-zero coordinates are processed). This sweep fixes `heap_factor`= 0.05 and msb= 150, and evaluates mqt from 0 to 1200 at multiple *md* levels. Figure 4 shows nearly flat lines for all tested *md* values: recall and latency changes are negligible across the whole mqt range. This behavior indicates that, in these runs, the document budget and block limits dominate first; by the time low-weight query coordinates are reached, either useful candidates are already found or the active budget is already close to saturation.

Max Search Blocks (msb). msb limits the number of `InvertedBlock`s evaluated per query coordinate. This sweep fixes `heap_factor`= 0.05 and mqt= 0. Unlike mqt, msb has a strong effect at low values (Figure 4). For example, at $md = 100000$, recall rises from about 0.58 (msb= 10) to about 0.90 (msb$\approx$ 90) and to about 0.926–0.928 for msb in the 150–200 range, with only small latency changes around 51–52 ms once msb exceeds 70. Overall, the practical operating point is msb around 150 (as used in the baseline), where recall is already near saturation and additional blocks bring limited gains for this workload.

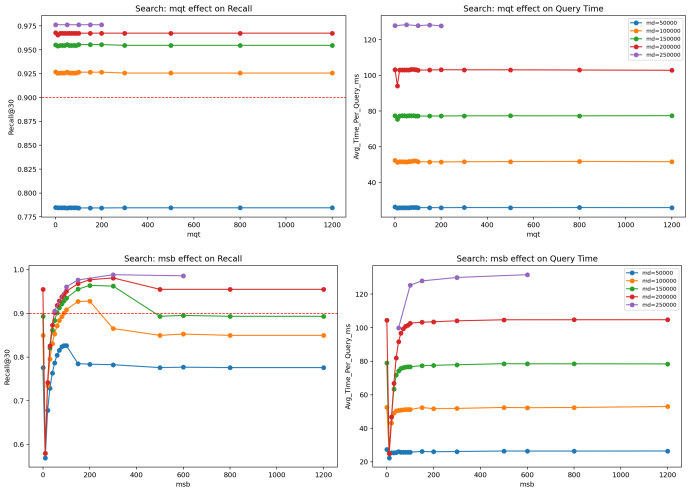

Fig. 4. Search controls at fixed index: top, mqt has near-zero effect because *md* binds first; bottom, msb strongly affects recall at low values and saturates for msb ≥ 150.

Heap Factor. Across the tested configurations, `heap_factor` has only minor impact compared with *md* and msb. In the selected baseline, *md* remains the dominant stopping condition, so changing `heap_factor` rarely alters the candidate set before the document budget is reached. We therefore keep `heap_factor`= 0.05 as a stable default.

4 Conclusion

This paper presents a mathematically grounded decoupled architecture that combines exact-maximum cluster summaries with CaaT traversal and block-level dynamic pruning. The static phase preserves a high-recall candidate pool, and the parameterized search phase provides direct runtime control of latency. Across the reported experiments, the `InvertedBlock` structure proves to be an effective data structure for approximate MIPS over LSR, since it consistently supports high recall while enabling practical latency control through online constraints.

Future work should further investigate this structure under broader workloads and larger tuning spaces to quantify its full potential and limits. Two immediate directions are automated search-parameter selection from query density statistics, and memory-aligned forward-index layouts to accelerate sequential access during exact sparse dot products.

Acknowledgments. The author thanks Eneko for the encouragement and motivation to participate in the SISAP indexing challenge. This work was partially funded by the ALBCOM research group at Universitat Politècnica de Catalunya (UPC), which supported attendance at the conference.

Disclosure of Interests. The author has no competing interests to declare that are relevant to the content of this article.

References

1. Broder, A. Z., Carmel, D., Herscovici, M., Soffer, A., Zien, J.: Efficient query evaluation using a two-level retrieval process. In: *Proceedings of the 12th International Conference on Information and Knowledge Management (CIKM)* (2003)
2. Formal, T., Piwowarski, B., Clinchant, S.: SPLADE: Sparse lexical and expansion model for first stage ranking. In: *Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval* (2021)
3. Thakur, N., Reimers, N., Rückle, A., Srivastava, A., Gurevych, I.: BEIR: A heterogeneous benchmark for zero-shot evaluation of information retrieval models. In: *NeurIPS Datasets and Benchmarks Track* (2021)
4. Mackenzie, J., Trotman, A., Lin, J.: Wacky weights in learned sparse representations and the revenge of score-at-a-time query evaluation. *arXiv preprint arXiv:2110.11540* (2021)
5. Roşca, G., Zamani, H., Makhervaks, M., et al.: Understanding wacky weights: A dissection of SPLADE's learned term importance. *arXiv preprint arXiv:2605.19628* (2026)
6. Bruch, S., Nardini, F. M., Rulli, C., Venturini, R.: Efficient inverted indexes for approximate retrieval over learned sparse representations. In: *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval* (2024)
7. Bruch, S., Nardini, F. M., Rulli, C., Venturini, R.: Pairing clustered inverted indexes with κ-NN graphs for fast approximate retrieval over learned sparse representations. In: *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management (CIKM)* (2024)